School of Electrical Engineering, Belgrade
Department of Computer Engineering and Information Theory

Course: Object-Oriented Programming (1)
Academic year: 2025/2026
(This assignment applies from the February 2026 examination period.)

Project Assignment for Homework

Document version: 1.0

English translation - original Serbian assignment is authoritative.

Contents

Contents **2**

Introduction **3**

Functional Requirements **4**

- **Basic Interpreter Behavior** **4**
- **Lexical Rules** **4**
- **Command Input and Output Streams** **5**
- **Commands** **7**
  - The echo command 7
  - The prompt command 7
  - The time command 7
  - The date command 8
  - The touch command 8
  - The truncate command 8
  - The rm command 8
  - The wc command 8
  - The tr command 9
  - The head command 9
  - The batch command 9
- **Pipelines** **9**
- **Error Handling** **10**

Other Requirements **11**

- **Scope of the Project Phases** **11**
- **Assessment Procedure** **11**

Revision History **13**

Introduction

The aim of this project is to implement a relatively simple command-line interpreter (CLI). Although considerably simplified compared with those used as interfaces in real operating systems (command-line interfaces, also abbreviated CLI), it includes many of the features and concepts supported by real operating systems. The interpreter must read commands one at a time from its character input and execute each command. Users enter commands interactively, but commands may also be supplied as a batch stored in a text file. The interpreter also supports redirection and pipelines, concepts commonly found in operating systems. All of these concepts and requirements are described in detail in this document. The interpreter must be implemented in C++.

This document specifies the project requirements for implementing the interpreter. Wherever this document does not define something precisely, the student should make a reasonable assumption, document it clearly and precisely, and base the solution on that assumption.

Functional Requirements

Basic Interpreter Behavior

The command-line interpreter must run as an interactive program that interacts with the user through the console of the host operating system. The user enters characters with the keyboard, and the program displays its response as text: a sequence of characters printed in order on one or more lines, with line breaks represented by the newline character (`'\n'`). The program must read input characters from its standard input and write its output to its standard output.

The interpreter repeatedly performs the following basic procedure:

1. At the beginning of a new output line, the interpreter displays a command prompt to indicate that it is ready to read a command line. Initially, this is the '$' character, but it can be changed.

```
$
```

2. The user enters the command line as a sequence of characters terminated by a newline character (`'\n'`). In its basic form, a command line contains one command. Commands can have different forms; their exact syntax and the set of commands are specified later. The general form is:

```
command [-opt] [argument]
```
[1]

Here, *command* is the command name, *opt* is an option that may affect how it executes and depends on the command, and *argument* is a command argument whose meaning also depends on the command. For example, the following command counts the words (-w for word) in the text supplied as a quoted argument:

```
$ wc -w "Lorem ipsum dolor sit amet, consectetur adipiscing elit"
```

3. The interpreter executes the specified command[2]. Depending on its meaning, the command may write its result to the output as a sequence of characters, potentially spanning several lines:

```
$ wc -w "Lorem ipsum dolor sit amet, consectetur adipiscing elit"
8
```

4. After the commands on the command line have finished executing, the procedure repeats: the interpreter displays the prompt for a new command line.

Lexical Rules

A command line is subject to the following lexical rules:

1. A command line consists of a sequence of at most 512 characters of type char, terminated by a newline (`'\n'`). If no newline is encountered within these 512 characters (not counting the newline), all

[1] Square brackets denote optional parts and are not part of the command itself.

[2] Or several commands, in the more general case where multiple commands are specified on one command line, as explained later.

remaining characters are ignored, and the first 512 characters entered (excluding the newline) are taken as the command line.

2. Uppercase and lowercase letters are distinct.

3. The command name, its option, and its argument are separated by whitespace: sequences of adjacent characters, each of which is a space (' ') or a horizontal tab (`'\t'`).

4. An argument enclosed in quotation marks (") is treated as a single character sequence passed to the command without splitting it into words at whitespace. Inside quotation marks, no character other than a newline (`'\n'`) or a quotation mark (") has any special meaning.

5. The vertical bar ('|'), less-than ('<'), and greater-than ('>') characters have special meanings described later, unless they occur inside a quoted character sequence. In that case, they are treated as part of the sequence and have no special meaning to the interpreter.

If the interpreter encounters one or more characters that are not permitted at their positions under the lexical rules and command syntax described here, it must not execute the command line. The entire line is considered invalid, and none of its commands is executed. In this case, the interpreter must display an error message containing at least the position of the first invalid character encountered. The implementation may provide more informative messages, for example by printing an error message, then the complete command line, and a symbol (such as ^) below every invalid character to help the user locate them. For example:

```
$ wc& -w *"Lorem ipsum dolor sit amet" +?
Error - unexpected characters:
wc& -w *"Lorem ipsum dolor sit amet" +?
  ^    ^                           ^^
```

Command Input and Output Streams

Most commands (but not all) operate on character sequences taken from an input character stream, which generalizes the following cases:

1. By default, if a command has no argument, its input stream is the console, i.e. the keyboard: it reads the characters the user types while the command is executing. Entering the character represented in the standard library by the symbolic constant EOF (Ctrl+Z on Windows, or Ctrl+D on Unix-like systems) ends the input character sequence, and therefore the input stream. For example, after this command is started:

```
$ wc -w
```

the command waits for the user to type a character sequence. If the user types the following and then presses the key combination that ends the input stream[3]:

```
Lorem ipsum dolor sit amet
```

the command will display only the number 5.

[3] These characters are not displayed on the screen: that is not part of the defined behavior of this command, which writes only its result, rather than the characters from its input stream, to its output stream.

2. If the command has an argument enclosed in quotation marks ("), the entire character sequence between the quotation marks (excluding the quotation marks themselves) is its input stream. For example, after this command is started:

```
$ wc -w "Lorem ipsum dolor sit amet"
```

the command will display the number 5.

3. If the command has an argument that is not enclosed in quotation marks, it denotes the name of a text file (more precisely, its path relative to the current directory in which the interpreter runs). The contents of that file, interpreted as text, form the command's input stream. For example, after this command is started:

```
$ wc -w input.txt
```

and if input.txt contains:

```
Lorem ipsum dolor sit amet
```

the command will display only the number 5.

4. Only if the command has no argument defining its input stream as in the preceding case may its input stream be redirected to a text file, so that it reads the file contents in the same way as above. Input redirection is denoted by '<'. For example, after this command is started:

```
$ wc -w <input.txt
```

and if input.txt contains:

```
Lorem ipsum dolor sit amet
```

the command will display only the number 5.

Analogously, a command has an output character stream: the destination to which it writes the characters that make up its output or result. This can be:

1. By default, the output stream is the console, i.e. the screen. The command displays its result, and the interpreter then moves the cursor to a new line to accept the next command line. For example, after this command is started:

```
$ wc -w "Lorem ipsum dolor sit amet"
```

the command will display the number 5.

2. A command's output stream may be redirected to a text file so that its output is written to that file. If the file does not exist, the command creates it. If it already exists, all of its previous contents are deleted and replaced with the command's result. Output redirection is denoted by '>'. For example, after this command is started:

```
$ wc -w "Lorem ipsum dolor sit amet" >output.txt
```

output.txt will contain only the character 5.

If output redirection uses two adjacent '>' characters with nothing between them, the command's output is appended to the end of the output file; any existing contents are not deleted.

Redirection may appear only at the end of the complete command. A command whose argument defines its input stream cannot also redirect that input stream. The redirection symbols ('<', a single '>', or a double '>') may, but need not, be preceded or followed by one or more whitespace characters under the lexical rules above. Subject to the

conditions stated for input and output redirection, both streams of a command may be redirected. The redirections may appear in either order after the command.

A command also has an error output stream: the destination to which it writes messages about errors that occur during its execution. By default, the error output stream is the console, i.e. the screen.

A filename is a sequence of adjacent characters terminated by a whitespace character under the lexical rules. The host operating system interprets this name when opening the file, so the character sequence must be passed to the system unchanged in the file-open operation.

Commands

The commands that must be supported are listed below. The system is intended to support future extensions with new commands: both commands with the general option-and-argument format described above, and commands with a more general format, allowing arbitrary additions after the command name, which always begins the command line. New options may also be introduced for existing commands.

The echo command

Format:

```
echo [argument]
```

Description: Passes characters from its input stream to its output stream without any changes.

Options: None.

The prompt command

Format:

```
prompt argument
```

Description: Changes (sets) the interpreter's prompt for reading a new command line to the character sequence supplied as a quoted argument.

Options: None.

The time command

Format:

```
time
```

Description: Writes the current time from the system real-time clock to its output character stream.

Options: None.

The date command

Format:

```
date
```

Description: Writes the current date from the system real-time clock to its output character stream.

Options: None.

The touch command

Format:

```
touch filename
```

Description: Creates an empty file with the specified name in the current directory (the directory in which the interpreter is running). If that file already exists, it prints an error message and has no other effect.

Options: None.

The truncate command

Format:

```
truncate filename
```

Description: Deletes the contents of the file with the specified name in the current directory.

Options: None.

The rm command

Format:

```
rm filename
```

Description: Deletes the file with the specified name (removes it from the file system) in the current directory.

Options: None.

The wc command

Format:

```
wc -opt [argument]
```

Description: Counts words or all characters in the text read from its input character stream and writes the count to its output stream. Words are sequences of adjacent characters separated by whitespace. For this command, whitespace is any character for which the standard-library function std::isspace returns true.

Options:
-w Counts words
-c Counts all characters

The tr command

Format:

```
tr [argument] -what [with]
```

Description: In the text read from its input stream, finds all occurrences of the quoted character sequence what, preceded by a - sign, and replaces them with the quoted character sequence with. It writes the resulting text to its output stream. If with is omitted, occurrences of what are simply removed from the input text.

Options: None.

The head command

Format:

```
head -ncount [argument]
```

Description: Copies the first few lines of the text read from its input stream to its output stream and ignores the remaining input.

Options: The mandatory -n option specifies the number of initial lines to copy from input to output. The characters -n are followed immediately (with no intervening characters) by a sequence of at most 5 decimal digits specifying the number of lines.

The batch command

Format:

```
batch filename
```

Description: Interprets the contents of the named input file as an arbitrarily long sequence of command lines separated by newline characters, executing all of them as described in this document, just as if they had been read from the console (batch processing). It interprets each line independently, one at a time, until the end of the input file. Any command without an argument, except commands that are not first in a pipeline, reads the required argument from the console, after which execution of the remaining commands continues. The commands' default output streams are the output stream of batch, but commands may specify individual output redirections. If a command line causes an error, its error message is written to the error output stream, and processing continues with the next line. The input file may contain any command, including batch itself; recursion must not be prevented, and its consequences are the user's responsibility.

Options: None.

Pipelines

A command line may contain several commands connected in a pipeline: the output stream of one command becomes the input stream of the next command on the line. Each character written by the first command is thus passed to the second command, preserving character order. Each command uses its defined format. The pipeline connection symbol is the vertical bar ('|').

For example, the following command line contains three commands connected in a pipeline:

```
time | tr -":" "." | wc -c > time.txt
```

The output stream of the first command, time, is connected to the input stream of the second command, tr, so all characters output by time are received by tr. The second command replaces the character ':' with '.', and the transformed text is then passed to the input of the third command, wc. That command counts all characters in the text and writes the result to text.txt.

A command whose input or output stream is connected to a pipeline cannot have an argument that defines that same stream in another way (as literal text, a file, or by redirection). The pipeline symbol may, but need not, be surrounded by whitespace.

Commands without an input stream (such as time and date) may appear only as the first command of a pipeline. Commands without an output stream may appear only as the last command of a pipeline.

Error Handling

Any error encountered during lexical or syntactic analysis, or during command execution, must result in an error message being written to the output stream and the command being abandoned without any effect. Errors include the following categories:

1. Lexical errors are handled as described earlier.

2. Unknown command. This error must produce the message `Unknown command: command`, showing the character sequence found where an expected command was not recognized.

3. Syntax errors in the format of a command or the entire command line.

4. Semantic errors in the definition of an input or output character stream: for example, a command with both an argument and input redirection, or a command whose input or output stream is connected to a pipeline and is also defined in another way.

5. A semantic error during command execution, where specifically defined for an individual command.

6. Errors reported by the operating system during command execution, for example while working with files (a file that is expected to exist does not exist, an access-permission error, or any other error).

Other Requirements

Scope of the Project Phases

The first project phase must implement the following subset of requirements:

- Execution of only the echo, time, date, touch, and wc commands.
- Support for arguments and all options of those commands, including input from a literal quoted argument or a text file.
- Handling only error types 5 and 6 listed in the "Error Handling" section. All test cases in this phase will be lexically, syntactically, and semantically correct, except that they may cause errors of types 5 and 6.
- Input and output redirection and pipelines need not be supported.

The second project phase must implement all requirements stated in this document.

Assessment Procedure

To be assessed, the project must compile and link without any errors. A project with a compilation or linking error will be rejected without further assessment. Resolving all compiler warnings is recommended.

To be assessed, the program must pass all public tests without errors. A program that fails a public test will be rejected without further assessment. Students will have access to the public tests and their correct results while developing the project.

After passing the public tests, the program will be tested using hidden tests (not known in advance). If the program fails any of the public tests, the project may be rejected entirely or receive a penalty in points, depending on the type of test and the severity of the error.

After the hidden tests, provided the project has not already been rejected or awarded zero points, an examiner will review its quality. The following aspects will be considered:

1. Programming style and presentation:
   - Indentation.
   - Consistency of style.
   - Clarity and consistency of naming (identifiers and letter case).
   - Clear separation of logical units in the source code (appropriate blank lines separating logical units in class definitions, subroutine bodies, and files).
   - Comments in appropriate places (neither too many nor too few).
2. Object decomposition:
   - Abstraction: clear and well-defined concepts (abstractions and classes).
   - Distribution of responsibilities among classes: well-defined and clear responsibilities, allocated according to the principles of high internal cohesion and low external coupling.
   - Modularity: decomposition into modules (files), with clear and well-defined module interfaces (headers and their contents).

- Class-level encapsulation: clear and well-defined class interfaces and implementations.
- Hierarchical decomposition: well-defined class hierarchies and reasonable use of inheritance.
- Collaboration mechanisms: well-defined and easily understood interaction mechanisms, with collaboration scenarios that are easy to follow.
- Relationships between classes: clear dependencies between classes and links between objects.
- Algorithmic decomposition: appropriate division into subroutines with short bodies.
- Overall comprehensibility of the project and ease of maintenance (modification and extension).

3. Object-oriented programming technique and command of C++

This category assesses the correct use of OOP and C++ concepts and techniques covered in the course. This does not mean that a program should include every such concept and technique. On the contrary, using a concept or technique where it is inappropriate or unnecessary may be assessed negatively. Students should not use everything they have learned merely to demonstrate it, but should use concepts and techniques where they are necessary and justified, with a sound explanation for their choice. The concepts and techniques assessed include everything covered in class, for example (this list is not exhaustive):

- Classes and class members.
- Class derivation (single/multiple; public/protected/private).
- Operator overloading.
- Pointers and references.
- Const-correctness.
- Constructors.
- Managing objects that own allocated resources (copy/move constructors, copy/move assignment operators, and destructors).
- Memory allocation management.
- Exception handling.
- Performance.

and similar topics.

Revision History

Version 1.0

Pages	Changes
All	Document written